Data Visualization & Analytics Lab

Experiment No. : 07

Implementation of Logistic Regression using Python.

Experiment No.09

6. **Aim:** Implement **Logistic Regression** using **Python** and visualize the results.

7. **Objectives:**
   - To understand **logistic regression** for predicting categorical outcome.
   - To visualize the relationships between independent variables and predicted classes.

8. **Course Outcomes**: Students will be able to apply and analyse the descriptive data analytics technique **for prediction.**

9. **Hardware / Software Required:** Python

10. **Theory:**

Logistic Regression is a statistical method used for **binary or multi-class classification** problems. Unlike linear regression, the output is categorical.

The logistic function (sigmoid function) is:

$$P(Y=1) = \frac{1}{1+e^{-(b_0+b_1X_1+\ldots+b_nX_n)}}$$

Where:

- **Y** → Dependent categorical variable (e.g., 0 or 1)
- **$X_1, X_2 \ldots X_n$** → Independent variables
- **$b_0, b_1 \ldots b_n$** → Coefficients

Steps:

1. Import required libraries.
2. Load dataset (with categorical target variable).
3. Define independent (X) and dependent (Y) variables.
4. Split dataset into training and testing sets.
5. Create logistic regression model.

6. Train the model with training data.
7. Predict target values for test data.
8. Visualize data relationships and model predictions.
9. Evaluate performance using metrics like Accuracy, Confusion Matrix, Precision, Recall, and ROC curve.

Sample code:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report,
roc_curve, auc

# Load dataset
data = pd.read_csv("data.csv")

# Independent variables
X = data[['X1', 'X2', 'X3']]

# Dependent variable (categorical)
Y = data['Y']

# Split dataset
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Create logistic regression model
model = LogisticRegression()

# Train model
model.fit(X_train, Y_train)
```

```
# Predictions
predictions = model.predict(X_test)
print("Predicted Values:")
print(predictions)

# Performance Evaluation
accuracy = accuracy_score(Y_test, predictions)
conf_matrix = confusion_matrix(Y_test, predictions)
class_report = classification_report(Y_test, predictions)
print("\nAccuracy:", accuracy)
print("\nConfusion Matrix:\n", conf_matrix)
print("\nClassification Report:\n", class_report)

# Visualization
# Pairplot to visualize relationships
sns.pairplot(data, hue='Y')
plt.show()

# Correlation heatmap
sns.heatmap(data.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()

# ROC Curve
y_prob = model.predict_proba(X_test)[:,1]
fpr, tpr, thresholds = roc_curve(Y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.plot(fpr, tpr, color='blue', label='ROC curve (area = %0.2f)' % roc_auc)
plt.plot([0, 1], [0, 1], color='red', linestyle='--')
plt.xlabel('False Positive Rate')
```

```
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend(loc="lower right")
plt.show()
```

6. Conclusion:

The experiment demonstrated how logistic regression combined with visualization provides both numerical and visual insights for classification problems.

7. Viva Questions:

- What is logistic regression?
- What are the performance measures used to evaluate logistic regression model?

References:

1. https://www.kaggle.com/code/stieranka/logistic-regression-with-python
2. 9 Best Python Libraries For Data Visualization: https://www.index.dev/blog/python-libraries-for-data-visualization